

GitHound Sample BloodHound Cypher Queries

Paste these into BloodHound Explore after importing GitHound data. These assume the GitHub OpenGraph extension schema is registered and GitHound data has been ingested.

Sources:

- [GitHound Cypher Queries](#)
- [GitHound Getting Started](#)

GitHub Owners

```
MATCH p=( :GH_User)-[:GH_HasRole]->( :GH_OrgRole {short_name:'owners'})
RETURN p
LIMIT 1000
```

Organizations Without 2FA

```
MATCH (o:GH_Organization)
WHERE o.two_factor_requirement_enabled = false
RETURN o
LIMIT 1000
```

Public Repositories

```
MATCH (repo:GH_Repository {private: false})
RETURN repo
LIMIT 1000
```

Unprotected Default Branches

```
MATCH p=(repo:GH_Repository)-[:GH_HasBranch]-(branch:GH_Branch {protected: false})
WHERE repo.default_branch = branch.short_name
RETURN p
LIMIT 1000
```

Protected Branches Without PR Reviews

```
MATCH p=( :GH_BranchProtectionRule {required_pull_request_reviews: false})-
[:GH_ProtectedBy]->( :GH_Branch)
```

```
RETURN p
LIMIT 1000
```

Users Who Can Push To Protected Branches

```
MATCH p=(actor)-[:GH_RestrictionsCanPush]->(rule:GH_BranchProtectionRule)-
[:GH_ProtectedBy]->(branch:GH_Branch)
RETURN p
LIMIT 1000
```

Secrets Reachable By Users

Users with write access can create GitHub Actions workflows to access repository and organization secrets.

```
MATCH p=( :GH_User)-[:GH_HasRole|GH_HasBaseRole|GH_MemberOf*1..]->( :GH_RepoRole)-
[:GH_WriteRepoContents]->( :GH_Repository)-[:GH_HasSecret]->(s)
WHERE s:GH_RepoSecret
OR s:GH_OrgSecret
RETURN p
LIMIT 1000
```

Repos Vulnerable To Workflow Secret Exfiltration

Finds secrets reachable by users who can create new branches.

```
MATCH p1=( :GH_User)-[:GH_HasRole|GH_HasBaseRole|GH_MemberOf*1..]->( :GH_RepoRole)-
[:GH_CanCreateBranch]->(repo:GH_Repository)-[:GH_HasSecret]->(s)
WHERE s:GH_RepoSecret
OR s:GH_OrgSecret
OPTIONAL MATCH p2=(repo)<-[:GH_CanCreateBranch]-( :GH_User)
OPTIONAL MATCH p3=(repo)<-[:GH_CanCreateBranch]-( :GH_Team)<-
[:GH_HasRole|GH_MemberOf|GH_AddMember*1..]-( :GH_User)
RETURN p1, p2, p3
LIMIT 1000
```

Secret Scanning Disabled

```
MATCH (repo:GH_Repository {secret_scanning: 'disabled'})
RETURN repo
LIMIT 1000
```

Open Secret Scanning Alerts

```
MATCH p=(repo:GH_Repository)-[:GH_Contains]->(:GH_SecretScanningAlert
{state:'open'})
RETURN p
LIMIT 1000
```

Fine-Grained PATs With All Repo Access

```
MATCH p=(:GH_User)-[:GH_HasPersonalAccessToken]->(token:GH_PersonalAccessToken
{repository_selection: 'all'})
RETURN p
LIMIT 1000
```

GitHub To Azure OIDC Trusts

```
MATCH p=(src)-[:GH_CanAssumeIdentity]->(cred:AZFederatedIdentityCredential)
RETURN p
LIMIT 1000
```

Good First Run Order

1. GitHub Owners
2. Secrets Reachable By Users
3. Unprotected Default Branches
4. Repos Vulnerable To Workflow Secret Exfiltration
5. GitHub To Azure OIDC Trusts